

# 离散数学 2

## 目录

离散数学 2 ..... 1

一、图论基础概念 2026-02-25 ..... 1

    (1) 特别基础：图的顶点、边、方向 ..... 1

    (2) 握手定理 ..... 2

    (3) 赋权图 ..... 2

    (4) 特殊的图 ..... 2

        完全图：所有点之间互相连接 ..... 2

            无向完全图 ..... 2

            有向完全图 ..... 2

$k$ -正则图 ..... 2

        圈图 ..... 2

        轮圈 ..... 2

        二分图 ..... 2

        Ramsey 问题简单记一下 ..... 2

    (5) 子图 ..... 2

    (6) 同构 ..... 3

        同构的判断充要条件 ..... 3

    (7) 图的运算 ..... 3

二、图的代数结构 2026-03-06 ..... 3

    (1) 邻接矩阵 ..... 3

        转置 ..... 3

        乘积 ..... 3

    (2) 关联矩阵 ..... 3

        对于无向图， ..... 4

        对于有向图， ..... 4

    (3) 边列表 ..... 4

    (4) 正向表 ..... 4

    (5) 逆向表 ..... 5

    (6) 邻接表 ..... 5

    (7) 有向图的十字链表 ..... 5

### 一、图论基础概念 2026-02-25

#### (1) 特别基础：图的顶点、边、方向

图  $G$  主要由两部分组成：

1. 顶点集合  $V$ ，里面的东西是节点或者顶点
2. 边集合  $E$ ，是  $V$  上的二元关系，顶点集合的笛卡尔积上的多重集（即可以有重复的元素）
3. 表示方法

$$V = \{v_1, v_2, v_3, \dots, v_n\}$$

$$E = \{(v_1, v_2), \dots, (v_4, v_5)\}$$

#### 4. 对有限图 $G = (V, E)$

1. 阶  $= |V| = n$
2.  $|E| = 0$  空图
3.  $|V| = 1$  平凡图
4. 未说明是默认有限无向图 一些举例：食物链是有向图

关联同时存在是就是无向图，否则可能是有向图。

5. 无向图不等价于两边都有的有向图

5. 对无向图： 1.认为顶点作为端点时，和它的边有**关联**；普通边关联次数为 1，自环关联次数为 2，不相连关联次数为 0

2. 环、孤立点
3. **顶点相邻**：两个顶点有公共边，和一个点相邻的所有点组成它的邻点集
4. 边相邻：两条边有公共顶点
5. 重边
6. 几种分类：简单图：无重边无自环

多重图：有重边有自环 伪图：都有 7. 度数：所连边的数量，自环贡献两个度；或者就叫**与其相邻边的关联次数之和**

6. 有向图：
  1. 通常用  $\langle v_1, v_2 \rangle$
  2. 起点/直接前驱**邻接到**终点/直接后继，反之叫做**邻接于**
  3. 直接后继集也是**外邻集**
  4. 直接前驱集也是**内邻集**
  5. 度数：上面**加粗文**；有入度和出度
7.  $\Delta, \delta$ ：最大度最小度 8. 度数列（同样的有入度列、出度列）：

## (2) 握手定理

1. 任意图的所有顶点度数之和是边数的 2 倍，有向图的入度=出度=边数
2. **!!! 任意图的奇度顶点的个数一定是偶数**

有神秘证明，简单理解：度数之和必然为偶数，偶度顶点度数之和必然为偶数，所以奇度顶点度数之和必然为偶数，所以一定有偶数个。

eg.（抽象为图结构的一个例子）不存在具有奇数个面且每个面都具有奇数条棱的多面体。 **面->点；棱->边**

## 3. 一个非空简单图，一定存在度数相同的顶点

数学归纳法：不存在孤立点、存在一个、存在两个。显然存在两个已经解决了。不存在和存在一个不赖，因为每个点的度数取值范围总比此时的顶点数少 1，所以肯定有相同的。

## (3) 赋权图

$w(e)$  就是  $e$  的边权。

## (4) 特殊的图

**完全图**：所有点之间互相连接

**无向完全图**

$$|V_{K_n}| = n(n-1)/2$$

**有向完全图**

$$m = n(n-1), \Delta^+ = \delta^+ = \delta^- = \Delta^- = n-1, \Delta = \delta = 2(n-1)$$

**$k$ -正则图**

每个顶点度数均为  $k$ ，边数为  $m = kn/2$

**圈图**

圈图至少 3 个顶点，并且是 2-正则图。

**轮圈**

在圈图里面放一个点，和圈上的所有点相连。

**二分图**

两部分只要非空就可以。完全二分图： $K_{m,n}$  二分图可能有多种剖分方式。

**Ramsey 问题简单记一下**

本质解决方法就是画出来图，然后分析实线/虚线多边形的存在可能性。

## (5) 子图

1. 定义： $V' \subseteq V, E' \subseteq E$ ，此时  $G' \subseteq G$
2. 真子图： $V' \subset V, E' \subset E$
3. **支撑 / 生成子图**：在定义基础上， $V' = V$

4. **导出子图**: 包含某个点集内部所有**相连的边集** / 某个边集关联的**所有顶点集**。记作  $G[V'], G[E']$ 。

## (6) 同构

存在双射函数, 将节点重新编号, 一一对应。  $G_1 \cong G_2$

判断同构的一些必要条件:

- 扭曲变形后和原图一致
- 顶点数、边数相同
- 度数列无序相同
- 对应顶点关联集和邻域相同
- 存在同构的导出子图
- **内部存在一样的封闭图形**

eg. 4 阶 3 边无向简单图。

**根据握手定理**, 总度数为 6, 故最大顶点度数为 3, 奇度点偶数个。现在我们列出来度数列就行了, 然后大概枚举一下。

eg.  $K_4$  的生成子图按照边数划分, 有 1 1 2 3 2 1 1。这对称是因为——补图同构!

## 同构的判断充要条件

- 对于简单图, 当且仅当补图同构
- 当且仅当邻接矩阵可以通过交换行列得到
- 所有子图都**同构**

## (7) 图的运算

1. 交:  $G_1 \cap G_2 = (V_1 \cap V_2, E_1 \cap E_2)$
2. 并:  $G_1 \cup G_2 = (V_1 \cup V_2, E_1 \cup E_2)$
3. 差:  $G_1 - G_2 = (V_1, E_1 - E_2)$
4. 补:  $\overline{G} = (V, E(K_n) - E)$
5. 笛卡尔积:  $G_1 \times G_2 = (V_1 \times V_2, E)$ , 其中  $E$  中的边满足:  $(u_1, v_1), (u_2, v_2) \in E$  当且仅当  $u_1 = u_2$  且  $(v_1, v_2) \in E_2$  或者  $v_1 = v_2$  且  $(u_1, u_2) \in E_1$ 。
6. 自补图:  $G$  与  $\overline{G}$  同构
7. 对称差:  $G_1 \oplus G_2 = (V_1 \cup V_2, E_1 \oplus E_2)$
8.  $G - v$ : 删除顶点  $v$  和与之关联的边
9.  $G - e$ : 删除边  $e$
10.  $G + e$ : 添加边  $e$ , 如果  $e$  已经存在则不变
11.  $G + v$ : 添加顶点  $v$ , 如果  $v$  已经存在则不变

## 二、图的代数结构 2026-03-06

### (1) 邻接矩阵

对于有向图, 每行表示一个顶点的出边, 每列表示一个顶点的入边。对于无向图, 邻接矩阵是对称的。

对于简单图, 邻接矩阵中的元素是 0 或 1, 表示是否存在边。对于多重图, 元素可以是非负整数, 表示边的数量。

对于赋权图, 元素可以是边权。此时称为权矩阵, 为 0 时不连通。

### 转置

对于有向图, 邻接矩阵的转置对应于将图中的所有边的方向反转。

$$G' = (V, E'), E' = \{(v, w) | (w, v) \in E\}$$

### 乘积

邻接矩阵的乘积可以用来计算路径数量。例如,  $A^k$  的元素  $a_{ij}$  表示从顶点  $i$  到顶点  $j$  的长度为  $k$  的路径数量。

### (2) 关联矩阵

关联矩阵  $M$  是一个  $n$  行  $m$  列的矩阵, 其中  $n$  是顶点数,  $m$  是边数。

对于无向图，

如果边  $e_j$  连接顶点  $v_i$ ，则  $M_{ij} = 1$ ；否则为 0。2 表示自环。

性质：

1. 从列来看， $\sum_{i=1}^n b_{ij} = 2, j = 1, 2, \dots, m$
2. 从行来看， $\sum_{j=1}^m b_{ij} = d(v_i), i = 1, 2, \dots, n$
3. 握手定理： $\sum_{i,j} b_{ij} = 2m$
4. 某列某元素为 2： $e_j$  是自环。
5. 相同的两列： $e_j$  和  $e_k$  是重边。
6. 某行全 0： $v_i$  是孤立点。

对于有向图，

如果边  $e_j$  从顶点  $v_i$  出发，则  $M_{ij} = 1$ ；如果边  $e_j$  到达顶点  $v_i$ ，则  $M_{ij} = -1$ ；否则为 0。

设无环有向图  $D = \langle V, E \rangle$ ，令  $B(D) = (b_{ij})_{n \times m}$ ，满足

$$b_{ij} = \begin{cases} 1, & v_i \text{ 是边 } e_j \text{ 的起点} \\ 0, & v_i \text{ 与边 } e_j \text{ 无关} \\ -1, & v_i \text{ 是边 } e_j \text{ 的终点} \end{cases}$$

矩阵的问题：效率极低，无法利用稀疏矩阵的性质，无法直接反映图的结构特征。

怎么优化？

### (3) 边列表

边列表有 2 个  $m$  维向量  $A, B$  组成，若  $e_k = (v_i, v_j)$ ，则  $A(k) = i, B(k) = j$ 。

举例：对于图  $G = (V, E)$ ，其中  $V = \{v_1, v_2, v_3\}$ ， $E = \{(v_1, v_2), (v_2, v_3)\}$ ，则边列表为  $A = [1, 2]$ ， $B = [2, 3]$ 。

特点：

- 适合存储稀疏图，节省空间。 $\mathcal{O}(nm) \Rightarrow \mathcal{O}(m)$
- 不适合存储密集图，查询效率低。枚举前驱后继效率低。

### (4) 正向表

将节点从 0 到  $n-1$  编号，将边列表按照起始节点排序，每一个节点的直接后继对应  $B$  中连续的一段下标区间。 $A$  的第  $n+1$  项存储为  $m$ ，表示边的总数。

正向表 (Forward Star) 结构图解

| 顶点 $v$ | head[v] |
|--------|---------|
| 1      | 1       |
| 2      | 3       |
| 3      | 4       |
| 4      | 6       |
| 5      | 7 (哨兵)  |

| 下标 $i$ | 终点 $w$ | 权重 |
|--------|--------|----|
| 1      | $v_2$  | 5  |
| 2      | $v_3$  | 2  |
| 3      | $v_4$  | 8  |
| 4      | $v_1$  | 1  |
| 5      | $v_5$  | 3  |
| 6      | —      | —  |

区间对应（指针指向）：

- 顶点 1 的边区间：edges[1] 到 edges[3-1]
- 顶点 2 的边区间：edges[3] 到 edges[4-1]
- 顶点 3 的边区间：edges[4] 到 edges[6-1]

特点：

1.  $d^+(v_i) = A(i+1) - A(i)$
2.  $A(i) = \sum_{j=0}^{i-1} d^+(v_j)$
3. 占用空间变成了  $\mathcal{O}(n+m)$ ，适合存储稀疏图。
4. 查询效率高，枚举直接后继效率高。

5. 前驱查询效率低。

(5) 逆向表

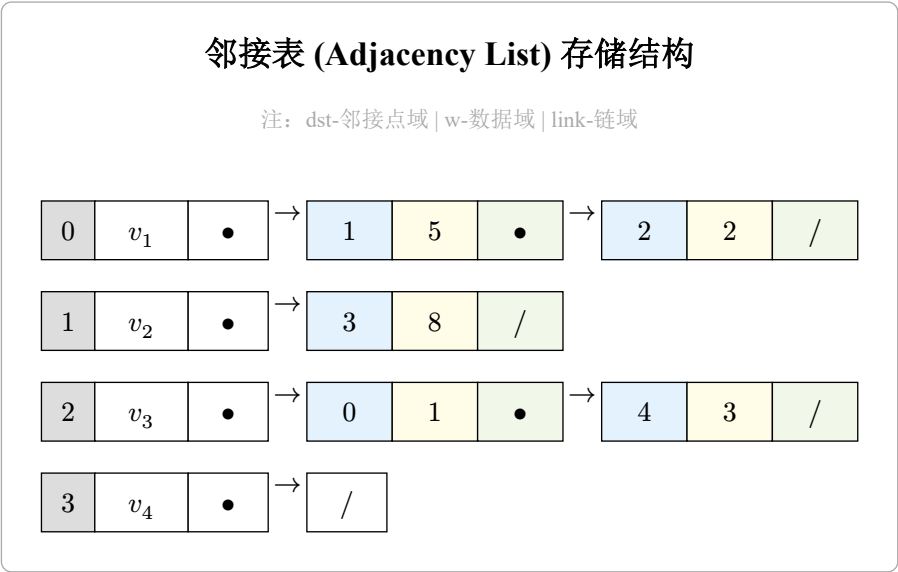
与正向表类似，但按照边的终点排序，每一个节点的直接前驱对应  $B$  中连续的一段下标区间。

无向图的逆向表和正向表是相同的。原因：无向图中边没有方向，正向表和逆向表的定义相同。

(6) 邻接表

每个节点维护一个链表，链表中存储与该节点相邻的边的信息（如终点和权重）。对于无向图，每条边会在两个节点的链表出现；对于有向图，每条边只会在起点的链表出现。起始节点只存储起始顶点。

- dst: 邻接点域
- w: 数据域
- link: 链域



表头节点结构

|      |           |
|------|-----------|
| data | firstedge |
|------|-----------|

边节点结构

|     |   |      |
|-----|---|------|
| dst | w | link |
|-----|---|------|

典型用途：查询某个节点的直接后继或前驱。复杂度为  $O(d^+(v))$  或  $O(d^-(v))$ 。

(7) 有向图的十字链表

节点结构如下：

- src: 边的起点
- dst: 边的终点
- weight: 边权
- d\_link: 指向同一起点的下一条边
- s\_link: 指向同一终点的下一条边

指针非常复杂。